

Розробка клієнта для взаємодії з віддаленою папкою з файлами. Етап 1, варіант 4-6

UML-специфікація системи MiniDrive

Виконав: студент групи МІ-41 Петров Богдан

Перевірив: _____

Київський національний університет імені Тараса Шевченка
Дисципліна «Інформаційні технології»
Київ — 2026

Зміст

1	Постановка задачі	2
2	Аналіз вимог	3
2.1	Дійові особи	3
2.2	Функціональні вимоги	3
2.3	Нефункціональні вимоги	4
2.4	Глосарій	4
2.5	Перелік прецедентів	5
3	Високорівневе проєктування	6
3.1	Архітектура	6
3.2	REST API	6
3.3	Рис. 1. Діаграма компонентів	7
3.4	Рис. 2. Діаграма розгортання	8
4	Деталізоване проєктування	9
4.1	Рис. 3. Діаграма прецедентів	9
4.2	Рис. 4. Діаграма класів — доменна модель	10
4.3	Рис. 5. VOPC-діаграма прецедента «Sort by name» + «Filter by type»	11
4.4	Рис. 6. Діаграма класів — сервер (NestJS)	11
4.5	Рис. 7. Діаграма класів — спільний пакет і клієнти	12
4.6	Рис. 8. Діаграма активності — синхронізація з локальною папкою (UC14)	14
4.7	Рис. 9. Діаграма активності — завантаження та оновлення файлу (UC9, UC10)	15
4.8	Рис. 10. Діаграма послідовності — реєстрація та вхід (UC1, UC2)	17
4.9	Рис. 11. Діаграма послідовності — завантаження та перегляд вмісту (UC9, UC8)	18
4.10	Рис. 12. Діаграма послідовності — синхронізація (UC14)	19
4.11	Рис. 13. Діаграма комунікації — сортування та фільтрація (UC5, UC6)	20

4.12 Рис. 14. Діаграма станів — сеанс клієнта	21
4.13 Рис. 15. Діаграма станів — життєвий цикл файлу під час синхронізації	22
4.14 Рис. 16. Діаграма станів — сеанс синхронізації (SyncEngine)	22

5 Інструменти	23
----------------------	-----------

6 Висновки	23
-------------------	-----------

1 Постановка задачі

Завдання модуля 1 — розробити програмну систему **типу 2**: клієнт для взаємодії з віддаленою папкою з файлами, полегшену версію Google Drive. Система отримала робочу назву **MiniDrive**.

Загальні вимоги до системи типу 2:

- авторизація на віддаленому сервері, після якої користувач опиняється у власному просторі-кабінеті (віртуальному диску);
- перегляд наявних файлів та їх атрибутів: назва, дата і час створення, дата і час зміни, ім'я того, хто завантажив, ім'я того, хто редагував;
- відображення або приховування всіх стовпців, крім стовпця з іменем;
- завантаження файлів на сервер, вивантаження та видалення файлів;
- синхронізація обраної папки на локальному диску з віддаленою;
- додаткові операції та типи файлів згідно з варіантом.

Варіант визначається двома останніми цифрами студентського квитка — **46**:

Цифра	Список	Значення для проєкту
4	ТИПИ файлів, вміст яких показується при кліку	.cs — як текст, .jpg — як зображення
6	ОПЕРАЦІЇ	сортування за назвою (зростання і спадання); фільтр «усі файли» / «лише .cpp, .png»

Етапи виконання та оцінювання: етап 1 — UML-специфікація (2 б.); етап 2 — десктоп-версія з unit-тестами та звітом (13 б.); етап 3 — веб-орієнтована версія (13 б.); етап 4 — порівняльний аналіз (2 б.). Бонуси: drag-and-drop при за/вивантаженні (+5 б.) та публікація серверної частини в інтернеті (+5 б.). Обидва бонуси заплановано.

Нюанс варіанта. Типи для перегляду (.cs, .jpg) і типи для фільтра (.cpp, .png) не збігаються, оскільки цифри варіанта незалежні. Прийнято проєктне рішення: перегляд реалізується узагальнено — будь-який текстовий файл (.cs, .cpp, .txt, .md, .json тощо) показується як текст, будь-яке растрове зображення (.jpg, .png, .gif) — як картинка. Обов'язковими й

перевіреними тестами випадками залишаються .cs та .jpg. Це рішення варто підтвердити з викладачем.

Метою етапу 1 є побудова UML-специфікації: діаграми прецедентів (1+), класів (2+, з-поміж них VOPC для прецедента варіанта), активності (1+), взаємодії (2+), станів (1+), компонентів (1+) і розгортання (1+).

2 Аналіз вимог

2.1 Дійові особи

Єдина дійова особа — **User** (користувач), особа з обліковим записом. Неавторизований відвідувач може лише зареєструватися або увійти. Сховище об'єктів (MinIO) і база даних (PostgreSQL) є внутрішніми компонентами системи, а не дійовими особами.

2.2 Функціональні вимоги

ID	Вимога	Прецеденти
R1	Реєстрація та вхід на віддаленому сервері; після входу — особистий простір (віртуальний диск)	UC1, UC2, UC3, UC15
R2	Перегляд файлів з атрибутами: назва, дата створення, дата зміни, хто завантажив, хто редагував (додатково — розмір і тип)	UC4
R3	Показ / приховування будь-якого стовпця, крім назви	UC7
R4	Сортування за назвою за зростанням і спаданням (операція варіанта)	UC5
R5	Фільтр: усі файли / лише .cpp і .png (операція варіанта)	UC6
R6	Клік по файлу показує вміст: .cs як текст, .jpg як зображення (тип варіанта)	UC8
R7	Завантаження файлу (нового або нової версії наявного), вивантаження, видалення	UC9, UC10, UC11, UC12

ID	Вимога	Прецеденти
R8	Синхронізація обраної локальної папки з віддаленим простором	UC13, UC14
R9	Бонус: drag-and-drop при завантаженні (обидва клієнти та вивантаженні (перетягування з вікна десктоп-клієнта)	UC9b, UC11a

2.3 Нефункціональні вимоги

- **Безпека:** паролі зберігаються як bcrypt-хеші; сесія — JWT (HS256, 24 год) у заголовку Authorization: Bearer; кожен користувач бачить лише файли свого простору.
- **Обмеження:** розмір файлу до 50 МБ; простір без підпапок (плоский список).
- **Переносимість:** десктоп-клієнт працює на macOS і Windows (Electron); веб-клієнт — у сучасних браузерях, синхронізація у вебі доступна в Chrome/Edge (File System Access API).
- **Розгортання:** сервер і сховище пакуються в Docker Compose і розгортаються як локально, так і на хмарній віртуальній машині.
- **Тестованість:** сортування, фільтр, вибір способу перегляду та план синхронізації — чисті функції у спільному пакеті, що покриваються unit-тестами без сервера.

2.4 Глосарій

Термін	Значення
Space	віртуальний диск — особистий простір користувача з файлами
FileEntry	метадані файлу: назва, розширення, розмір, дати створення/зміни, хто завантажив/редагував, ключ у сховищі
FileContent	вміст файлу як об'єкт у сховищі MinIO
FilePreview	абстракція перегляду; реалізації TextPreview (текст) та ImagePreview (зображення)
LocalFolder	прив'язана локальна папка на диску користувача
SyncPlan / SyncAction / SyncReport	план синхронізації (список дій upload / download / skip) та звіт про його виконання
SyncEngine	компонент десктоп-клієнта, який сканує папку, будує план і виконує його

2.5 Перелік прецедентів

ID	Прецедент	Опис
UC1	Sign up	реєстрація нового облікового запису
UC2	Log in to the system	вхід; необхідний для всіх подальших дій
UC3	Work with the file storage	кореневий прецедент роботи з кабінетом
UC4	View the list of files and their attributes	перегляд таблиці файлів з атрибутами
UC5	Sort by name	сортування за назвою: Ascending / Descending
UC6	Filter by type	фільтр: All files / Only .cpp, .png
UC7	Show / hide table columns	приховування стовпців (Creation date, Modification date, Uploaded by, Edited by, Size)
UC8	View the file contents	перегляд вмісту: .cs as text / .jpg as image
UC9	Upload file(s) to the storage	завантаження файлів кнопкою (Select and upload) або перетягуванням (Drag-and-drop)
UC10	Update a file (new version)	завантаження нової версії наявного файлу; оновлює дату зміни і поле «хто редагував»
UC11	Download file(s)	вивантаження; у десктоп-клієнті також перетягуванням з вікна (Drag out of the window)
UC12	Delete file(s)	видалення файлу з простору
UC13	Bind / change local folder	прив'язка або зміна локальної папки для синхронізації
UC14	Synchronize with the local folder	синхронізація з прив'язаною папкою; автоматичне відстеження змін (десктоп) та розв'язання конфліктів версій
UC15	Log out from the system	вихід із системи

3 Високорівневе проєктування

3.1 Архітектура

Система складається з трьох застосунків і спільного пакета в одному монорепозиторії (TypeScript):

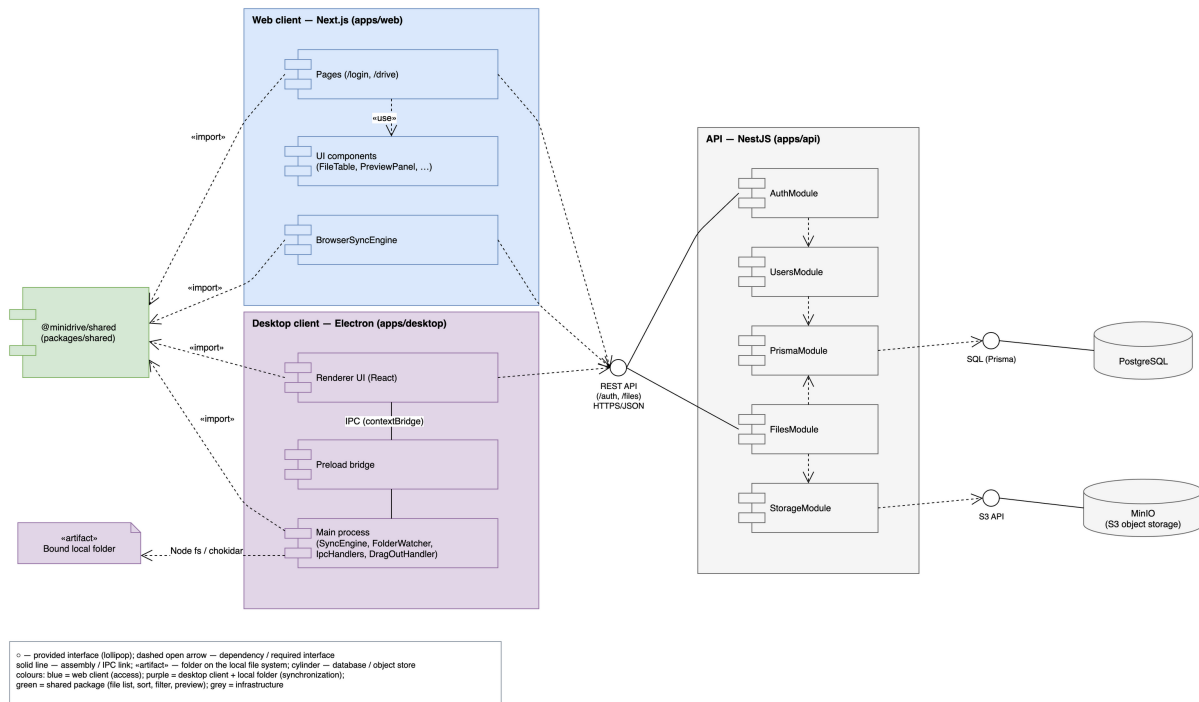
- **apps/api** — REST-сервер на NestJS: модулі AuthModule, UsersModule, FilesModule, StorageModule, PrismaModule; метадані у PostgreSQL (через Prisma), вміст файлів — у сховищі MinIO через S3-сумісний API.
- **apps/desktop** — десктоп-клієнт на Electron + React: головний процес (SyncEngine, FolderWatcher, IpcHandlers, DragOutHandler), preload-міст та рендерер з React-компонентами.
- **apps/web** — веб-клієнт на Next.js: сторінки /login та /drive, ті самі React-компоненти, BrowserSyncEngine на File System Access API.
- **packages/shared** — типи DTO, ієрархія FilePreview та чисті функції sortByName, filterByType, previewKindOf, toggleColumn, computeSyncPlan.

Обидва клієнти працюють з одним і тим самим REST API. Сортвання, фільтрація, видимість стовпців і план синхронізації обчислюються на клієнті функціями спільного пакета.

3.2 REST API

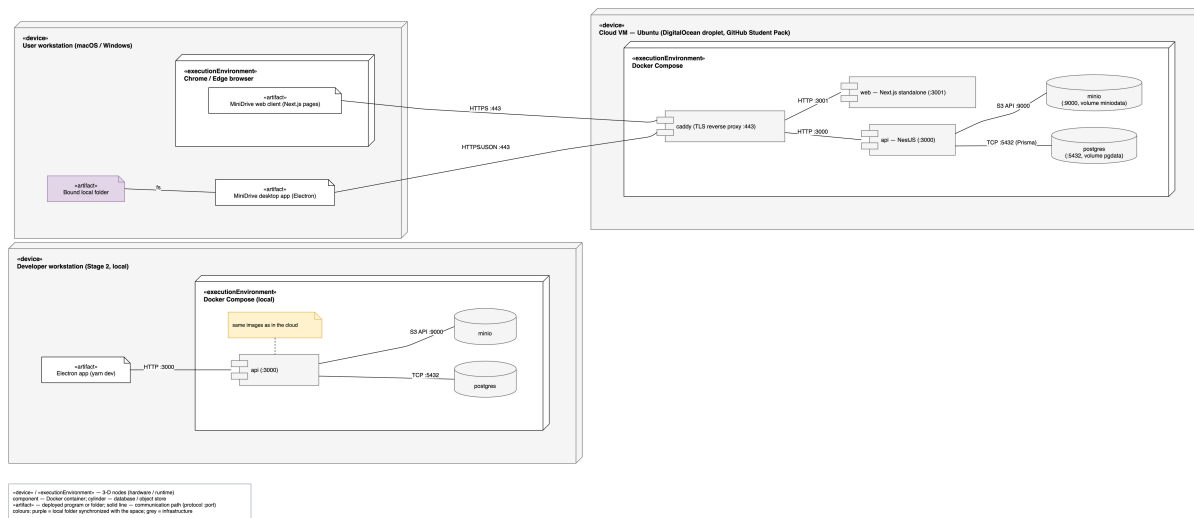
Метод	Шлях	Призначення
POST	/auth/register	реєстрація, повертає AuthResponseDto {accessToken, user}
POST	/auth/login	вхід, повертає AuthResponseDto
GET	/auth/me	поточний користувач
GET	/files	список FileDto[] простору користувача
POST	/files	завантаження (multipart); наявна назва — перезапис (нова версія)
GET	/files/:id/content	вміст файлу (перегляд і вивантаження)
DELETE	/files/:id	видалення

3.3 Рис. 1. Діаграма компонентів



Пояснення. Діаграма показує три підсистеми поверх спільного API. Десктоп-клієнт (Electron) складається з компонентів Main process, Preload bridge та Renderer UI (React), з'єднаних через IPC (contextBridge); головний процес працює з прив'язаною локальною папкою через Node fs / chokidar. Веб-клієнт (Next.js) містить сторінки /login і /drive, UI-компоненти та BrowserSyncEngine. Обидва клієнти імпортують спільний пакет @minidrive/shared і споживають інтерфейс REST API (/auth, /files) по HTTPS/JSON, який надає сервер NestJS з модулями AuthModule, UsersModule, FilesModule, StorageModule, PrismaModule. Сервер, своєю чергою, використовує інтерфейси SQL (Prisma) бази PostgreSQL та S3 API сховища MinIO. Діаграма фіксує межу перевикористання коду: React-компоненти і логіка списку файлів спільні, оболонка та доступ до файлової системи — окремі для кожного клієнта.

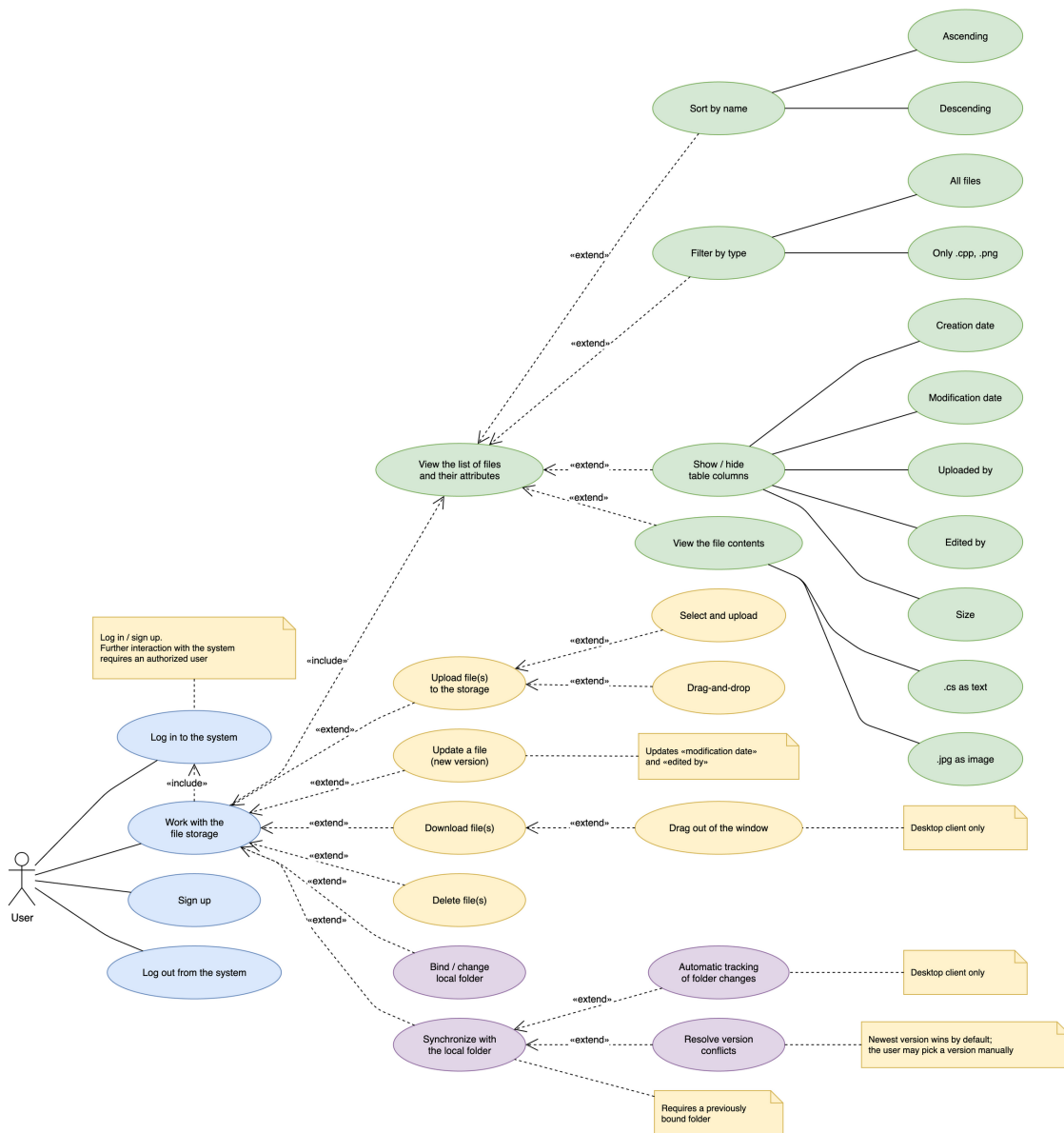
3.4 Рис. 2. Діаграма розгортання



Пояснення. Показано три вузли-пристрої. На робочій станції користувача розгорнуто артефакт «MiniDrive desktop app (Electron)», прив'язану локальну папку та браузер Chrome/Edge з веб-клієнтом. На хмарній віртуальній машині (Ubuntu, DigitalOcean droplet за програмою GitHub Student Pack) у середовищі виконання Docker Compose працюють контейнери caddy (TLS-проксі, порт 443), api (NestJS, порт 3000), web (Next.js standalone, порт 3001), postgres (порт 5432, том pgdata) і minio (порт 9000, том miniodata). Клієнти звертаються до сервера по HTTPS через caddy. Окремо показано робочу станцію розробника для етапу 2: той самий Docker Compose локально (api, postgres, minio) та Electron-застосунок у режимі розробки; примітка підкреслює, що використовуються ті самі образи, що й у хмарі.

4 Деталізоване проєктування

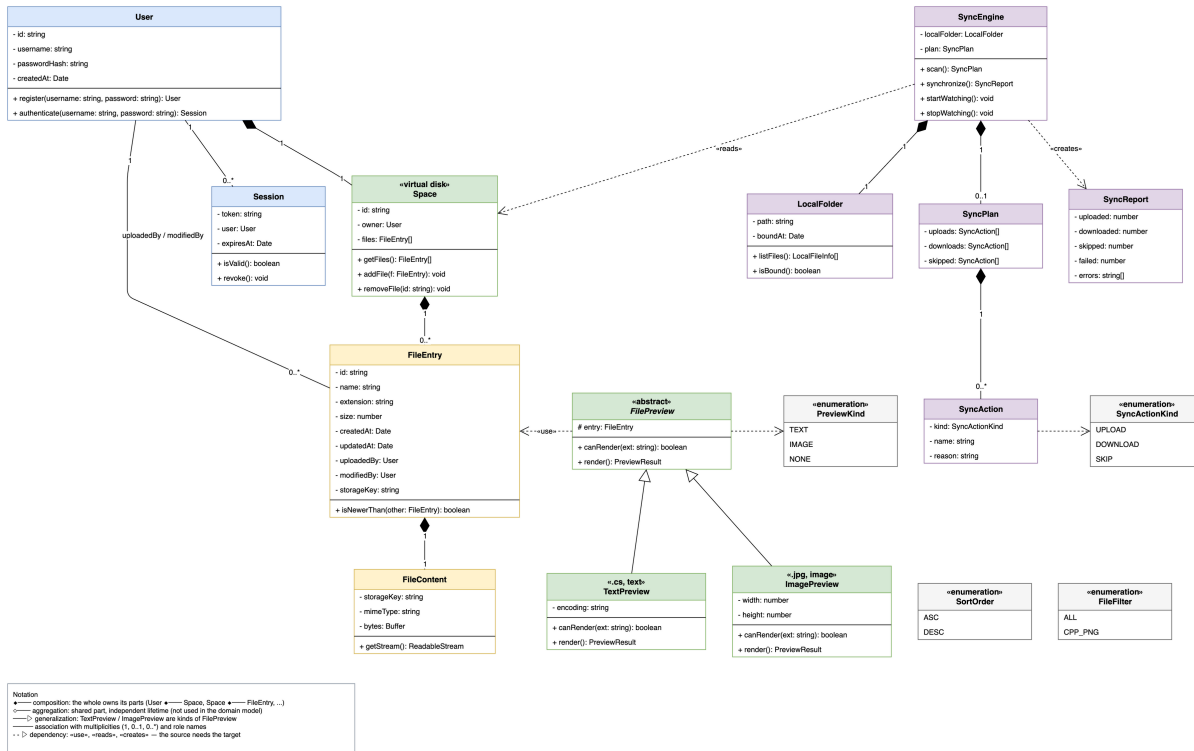
4.1 Рис. 3. Діаграма прецедентів



Пояснення. Діаграма побудована каскадом: дійова особа User пов'язана з корневими прецедентами Sign up, Log in to the system, Log out from the system та Work with the file storage. Корневий прецедент роботи зі сховищем через «include» вимагає входу в систему та перегляду списку файлів (View the list of files and their attributes). Від перегляду списку через «extend» відгалужуються операції варіанта — Sort by name з параметрами Ascending / Descending та Filter by type з параметрами All files / Only .cpp, .png, — а також Show / hide table columns (Creation date, Modification date, Uploaded by, Edited by, Size) і View the file contents (.cs as text, .jpg as image). Від кореня відгалужуються операції над файлами: Upload file(s) to the storage з підваріантами Select and upload та Drag-and-drop,

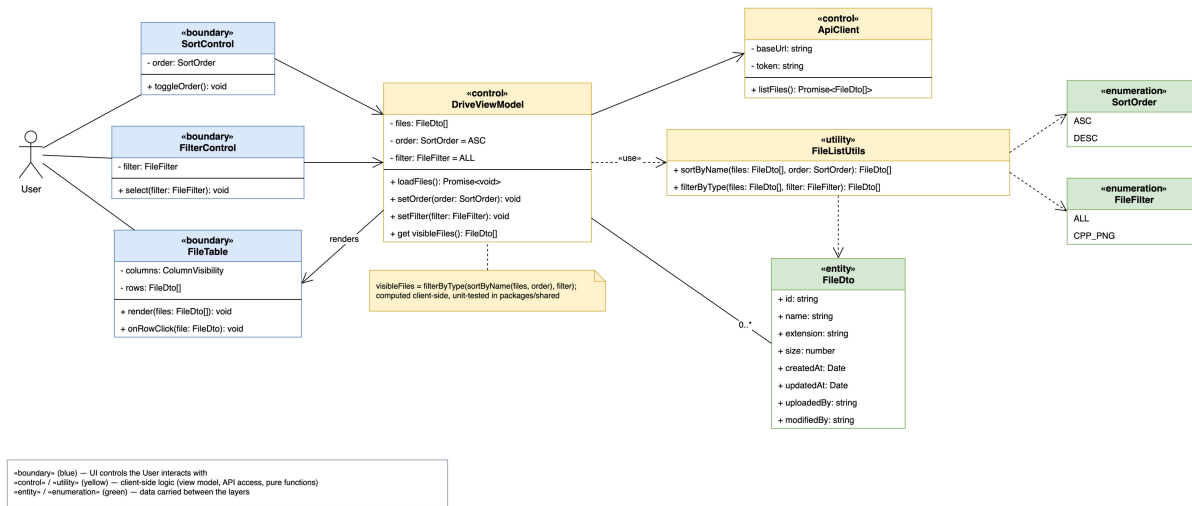
Update a file (new version), Download file(s) з підваріантом Drag out of the window, Delete file(s), а також гілка синхронізації: Bind / change local folder та Synchronize with the local folder з розширеннями Automatic tracking of folder changes і Resolve version conflicts. Примітки фіксують передумову авторизації, побічний ефект оновлення файлу (зміна дати та поля «хто редагував»), передумову прив'язаної папки, правило «перемагає новіша версія» та обмеження «лише десктоп-клієнт» для перетягування з вікна та автоматичного відстеження.

4.2 Рис. 4. Діаграма класів — доменна модель



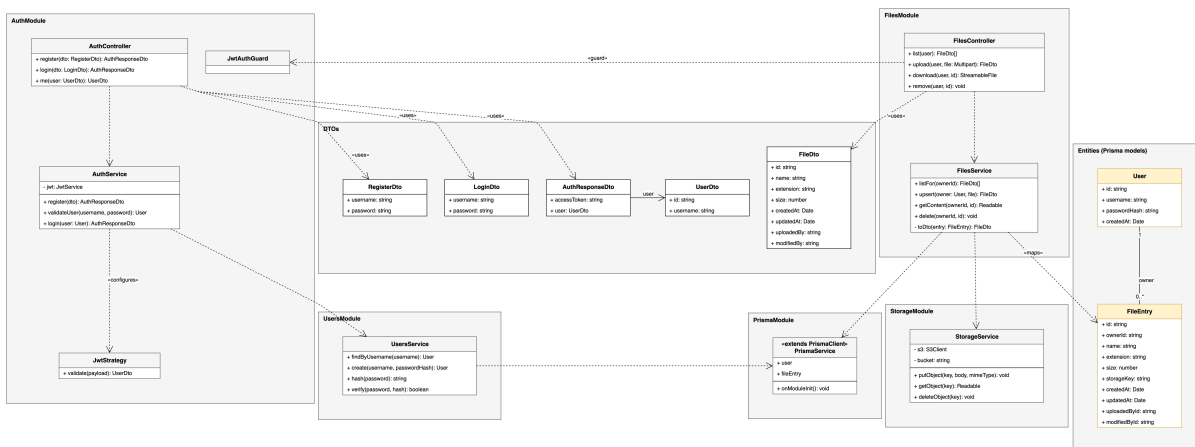
Пояснення. Доменна модель описує предметну область без прив'язки до фреймворків. User володіє одним простором Space («virtual disk»), який є композицією записів FileEntry; кожен запис має рівно один FileContent — об'єкт у сховищі. Асоціація FileEntry — User з полями uploadedBy / modifiedBy є джерелом атрибутів «хто завантажив» і «хто редагував». Session моделює JWT-сесію користувача (token, expiresAt, isValid()). Ієрархія перегляду: абстрактний FilePreview з операціями canRender(ext) та render() і два нащадки — TextPreview («.cs, text») та ImagePreview («.jpg, image»), що безпосередньо відбиває типи варіанта. Гілка синхронізації: SyncEngine володіє LocalFolder та планом SyncPlan, який складається з дій SyncAction (kind: UPLOAD | DOWNLOAD | SKIP), читає Space і створює SyncReport. Перелічення SortOrder {ASC, DESC} та FileFilter {ALL, CPP_PNG} кодують операції варіанта. Легенда пояснює позначення композиції, узагальнення, асоціації та залежності.

4.3 Рис. 5. VOPC-діаграма прецедента «Sort by name» + «Filter by type»



Пояснення. Діаграма класів, що беруть участь у прецеденті (View Of Participating Classes), побудована за методологією RUP зі стереотипами трьох ролей. Граничні класи «boundary» — SortControl (toggleOrder()), FilterControl (select(filter)) та FileTable (render(files), onRowClick(file)) — єдині, з якими взаємодіє User. Керівний клас «control» DriveViewModel зберігає поточні order: SortOrder і filter: FileFilter, приймає команди від граничних класів (setOrder, setFilter), завантажує список через «control» ApiClient.listFiles() і обчислює похідну властивість visibleFiles. Обчислення делеговано утилітному класу «utility» FileListUtils з чистими функціями sortByName(files, order) та filterByType(files, filter). Сутнісні класи «entity» — FileDto і перелічення SortOrder, FileFilter — є пасивними даними, які передаються між шарами. Примітка фіксує формулу visibleFiles = filterByType(sortByName(files, order), filter): обидві операції варіанта виконуються на клієнті та покриваються unit-тестами спільного пакета.

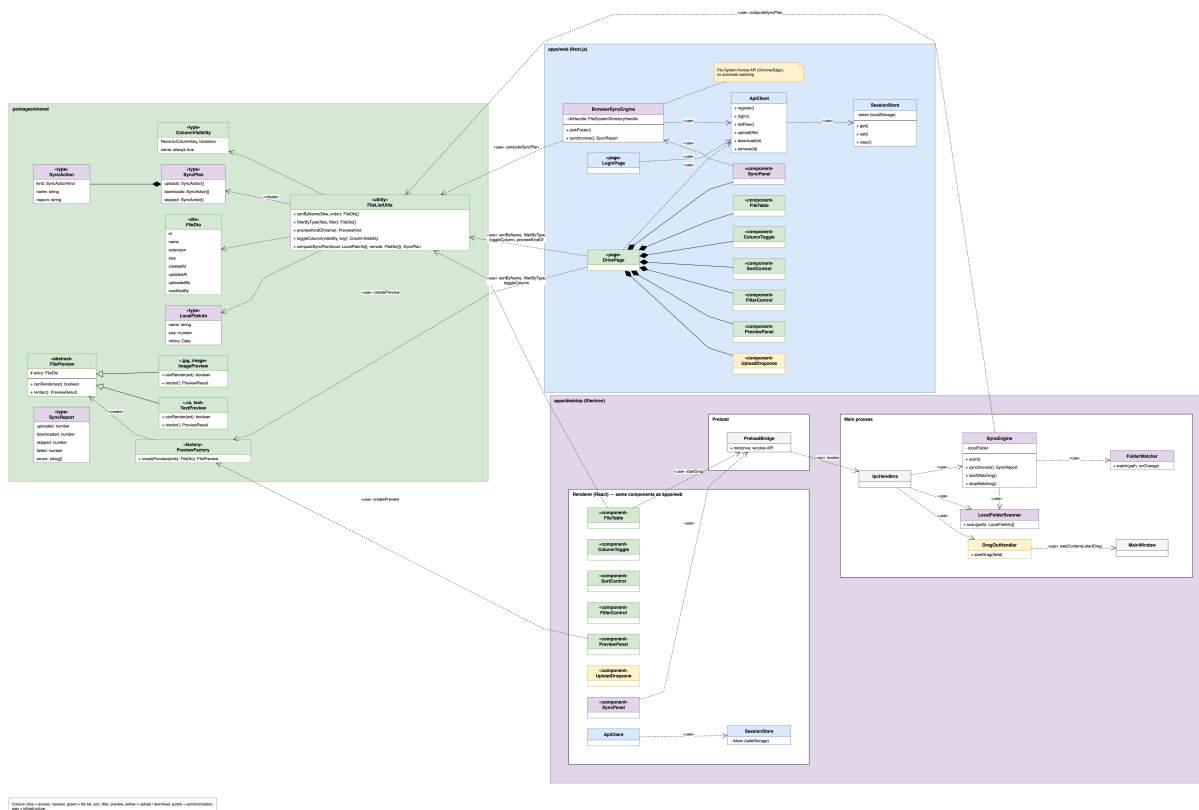
4.4 Рис. 6. Діаграма класів — сервер (NestJS)



Пояснення. Класи сервера згруповано за модулями NestJS. AuthModule містить AuthController

(register, login, me), AuthService (register, validateUser, login), JwtStrategy та охоронець JwtAuthGuard; UsersModule — UsersService з пошуком і створенням користувачів та хешуванням паролів; FilesModule — FilesController (list, upload, download, remove) і FilesService з операцією upsert(owner, file), яка реалізує перезапис файлу з наявною назвою; StorageModule — StorageService з операціями putObject, getObject, deleteObject над MinIO; PrismaModule — PrismaService, що розширює PrismaClient. Окремо показано DTO (RegisterDto, LoginDto, UserDto, AuthResponseDto, FileDto) та сутності Prisma User і FileEntry з асоціацією «owner» 1 — 0..*. Залежності «uses» / «guard» / «maps» показують, як контролери спираються на сервіси, а сервіси — на сховище та базу даних.

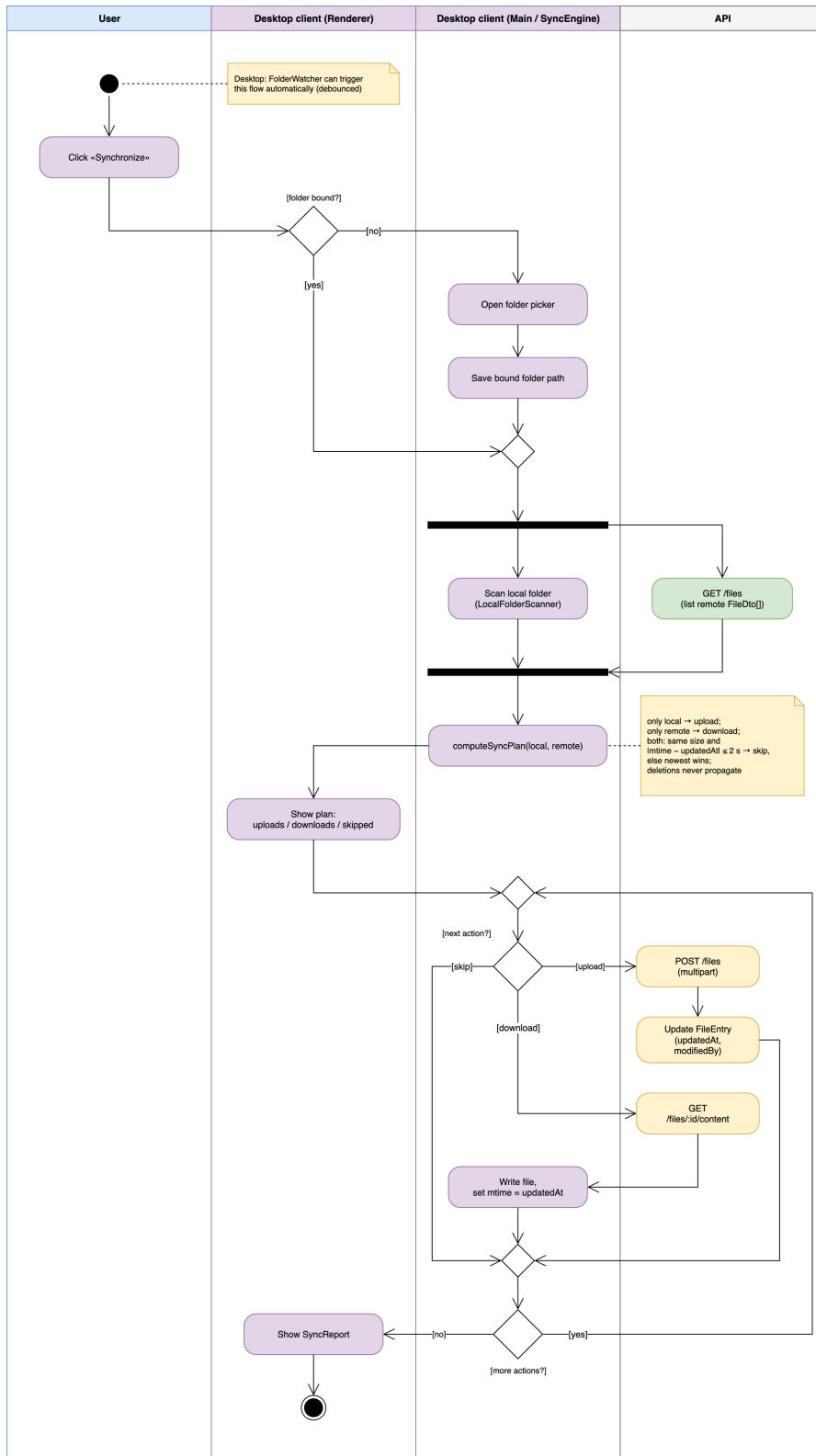
4.5 Рис. 7. Діаграма класів — спільний пакет і клієнти



Пояснення. Пакет packages/shared містить утилітний клас FileListUtils (sortByName, filterByType, previewKindOf, toggleColumn, computeSyncPlan), фабрику PreviewFactory.createPreview(entry), ієрархію FilePreview — TextPreview / ImagePreview та типи FileDto, LocalFileInfo, SyncPlan, SyncAction, SyncReport, ColumnVisibility. Пакет apps/web складається зі сторінок LoginPage, DrivePage, компонентів FileTable, ColumnToggle, SortControl, FilterControl, PreviewPanel, UploadDropzone, SyncPanel, а також ApiClient, SessionStore (токен у localStorage) і BrowserSyncEngine. Пакет apps/desktop поділено на три частини: головний процес (MainWindow, IpcHandlers, LocalFolderScanner, SyncEngine, FolderWatcher, DragOutHandler), preload (PreloadBridge) та рендерер, який переважно використовує ті самі React-компоненти. Залежності «use» показують, що обидва клієнти спираються на спільний

пакет, а ланцюжок `Renderer` → `PreloadBridge` → `IpcHandlers` → `SyncEngine` ілюструє, як команда синхронізації з інтерфейсу досягає файлової системи.

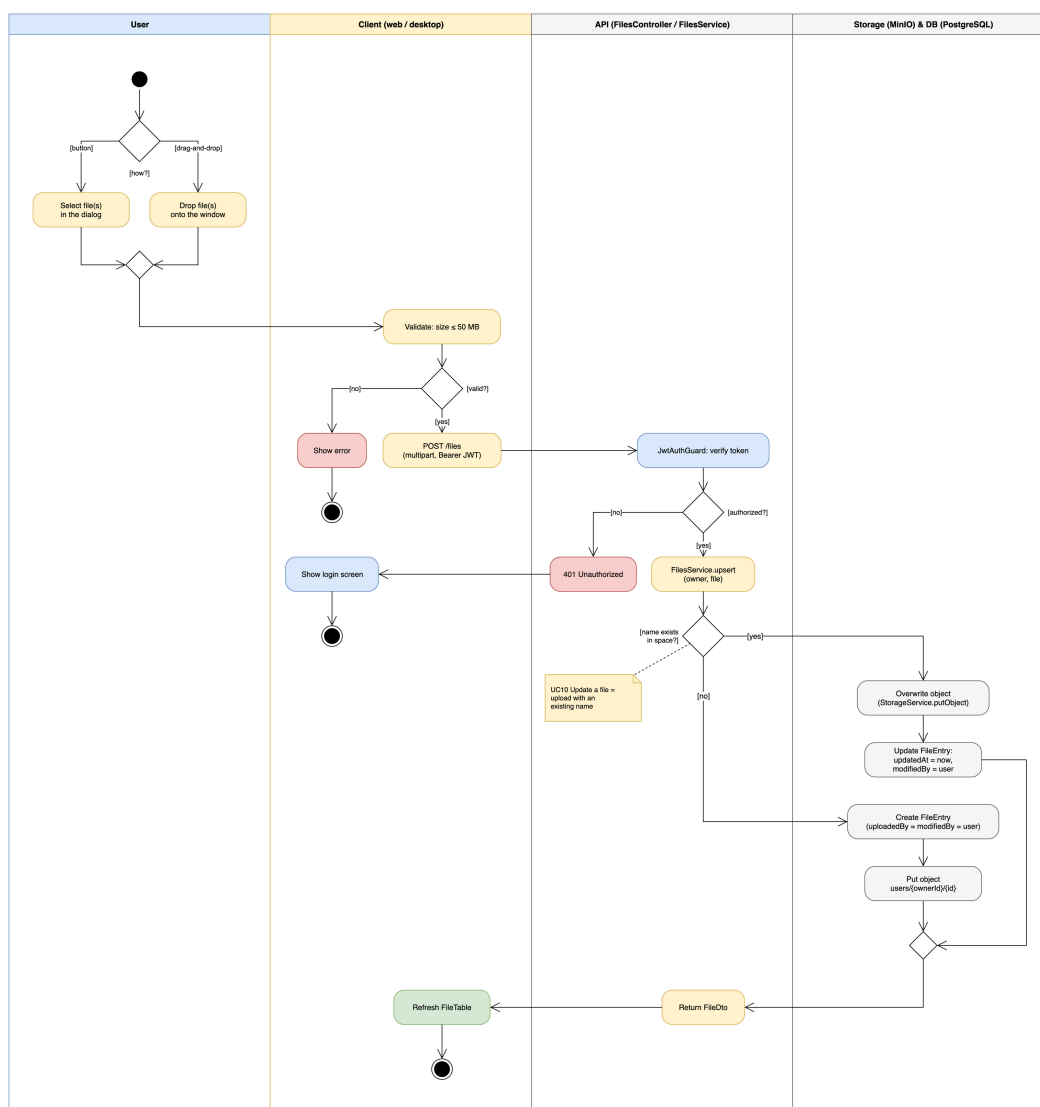
4.6 Рис. 8. Діаграма активності — синхронізація з локальною папкою (UC14)



Пояснення. Доріжки відповідають користувачеві, рендереру та головному процесу десктоп-

клієнта і серверу. Після натискання «Synchronize» перевіряється, чи прив'язано папку; якщо ні — відкривається діалог вибору, і шлях зберігається. Далі паралельно (вилка/злиття) виконуються сканування локальної папки (LocalFolderScanner) і запит GET /files. За обома списками обчислюється план computeSyncPlan(local, remote) за правилами з примітки: файл лише локально — завантажити; лише віддалено — вивантажити; в обох місцях з однаковим розміром і різницею часу зміни до 2 с — пропустити; інакше перемагає новіша версія; видалення ніколи не поширюються. Далі в циклі для кожної дії виконується POST /files (з оновленням updatedAt, modifiedBy), GET /files/:id/content із записом файлу та встановленням mtime, або пропуск. Після вичерпання дій показується SyncReport. Примітка на старті вказує, що FolderWatcher може запускати цей потік автоматично.

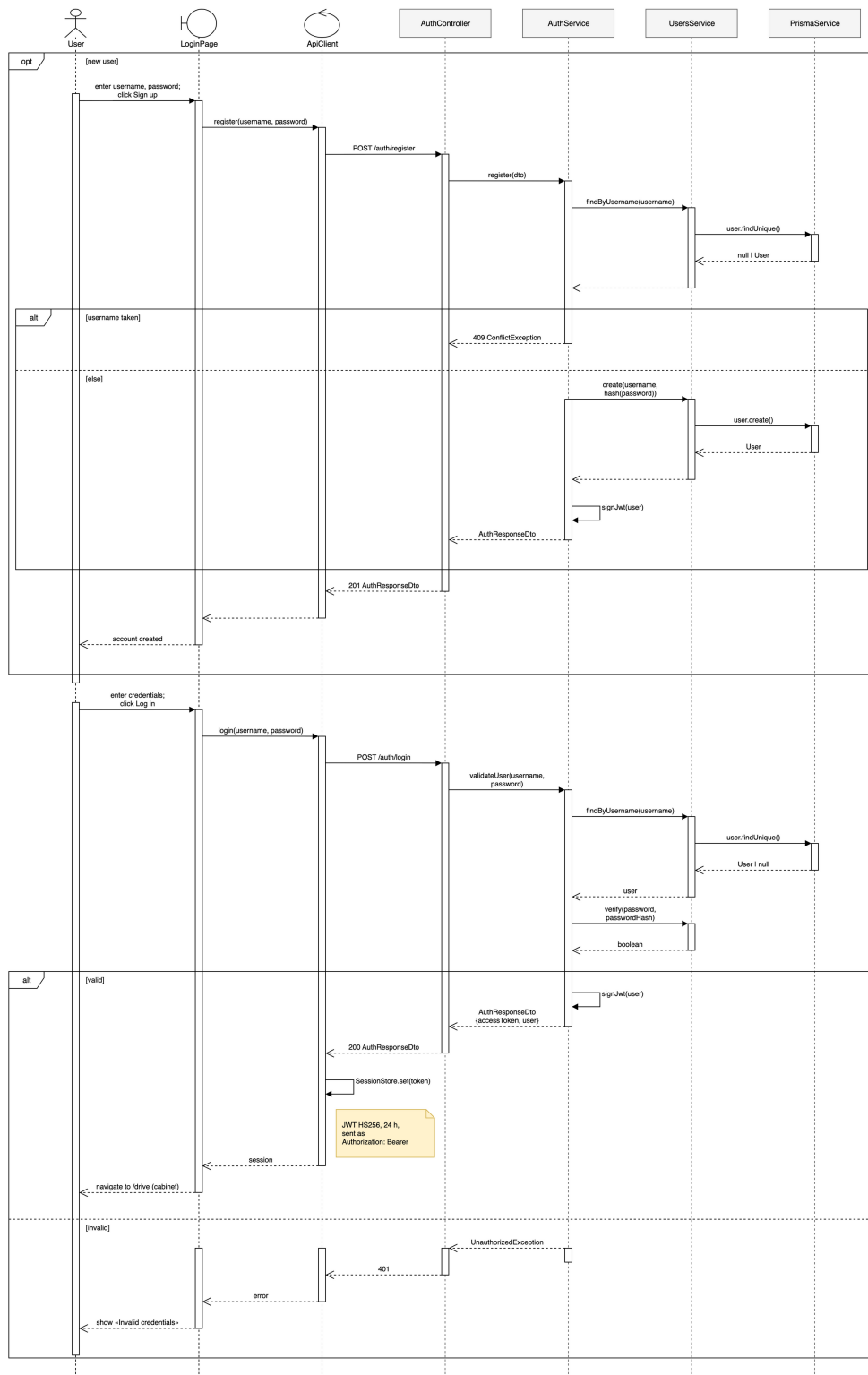
4.7 Рис. 9. Діаграма активності — завантаження та оновлення файлу (UC9, UC10)



Пояснення. Користувач обирає файли кнопкою або перетягує їх у вікно (бонус drag-and-drop);

обидві гілки зливаються. Клієнт перевіряє розмір (до 50 МБ) і надсилає `POST /files` з JWT. На сервері `JwtAuthGuard` перевіряє токен; у разі відмови повертається 401 і клієнт показує екран входу. Далі `FilesService.upsert(owner, file)` перевіряє, чи існує файл із такою назвою у просторі: якщо так — об'єкт перезаписується через `StorageService.putObject`, а в `FileEntry` оновлюються `updatedAt` і `modifiedBy` (це і є прецедент UC10 «Update a file»); якщо ні — створюється новий `FileEntry` з `uploadedBy = modifiedBy = user` і об'єкт кладеться за ключем `users/{ownerId}/{id}`. Сервер повертає `FileDto`, клієнт оновлює таблицю.

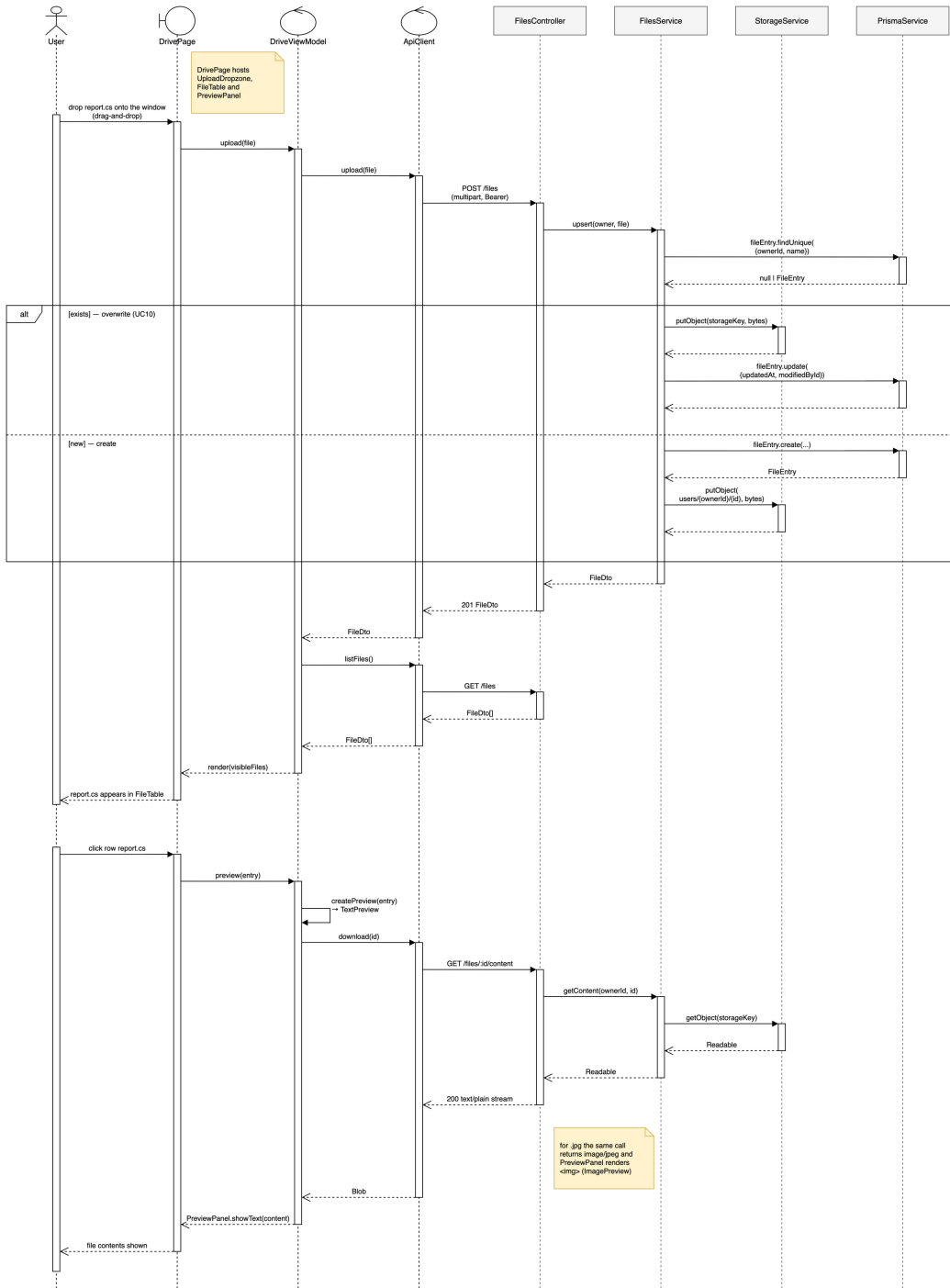
4.8 Рис. 10. Діаграма послідовності — реєстрація та вхід (UC1, UC2)



Пояснення. У фрагменті opt [new user] користувач через граничний об'єкт LoginPage і керівний ApiClient викликає POST /auth/register; AuthController передає запит до AuthService, який через UsersService і PrismaService перевіряє унікальність імені (alt: 409 ConflictException або створення користувача з хешем пароля і підписання JWT). Далі показано вхід: POST /auth/login,

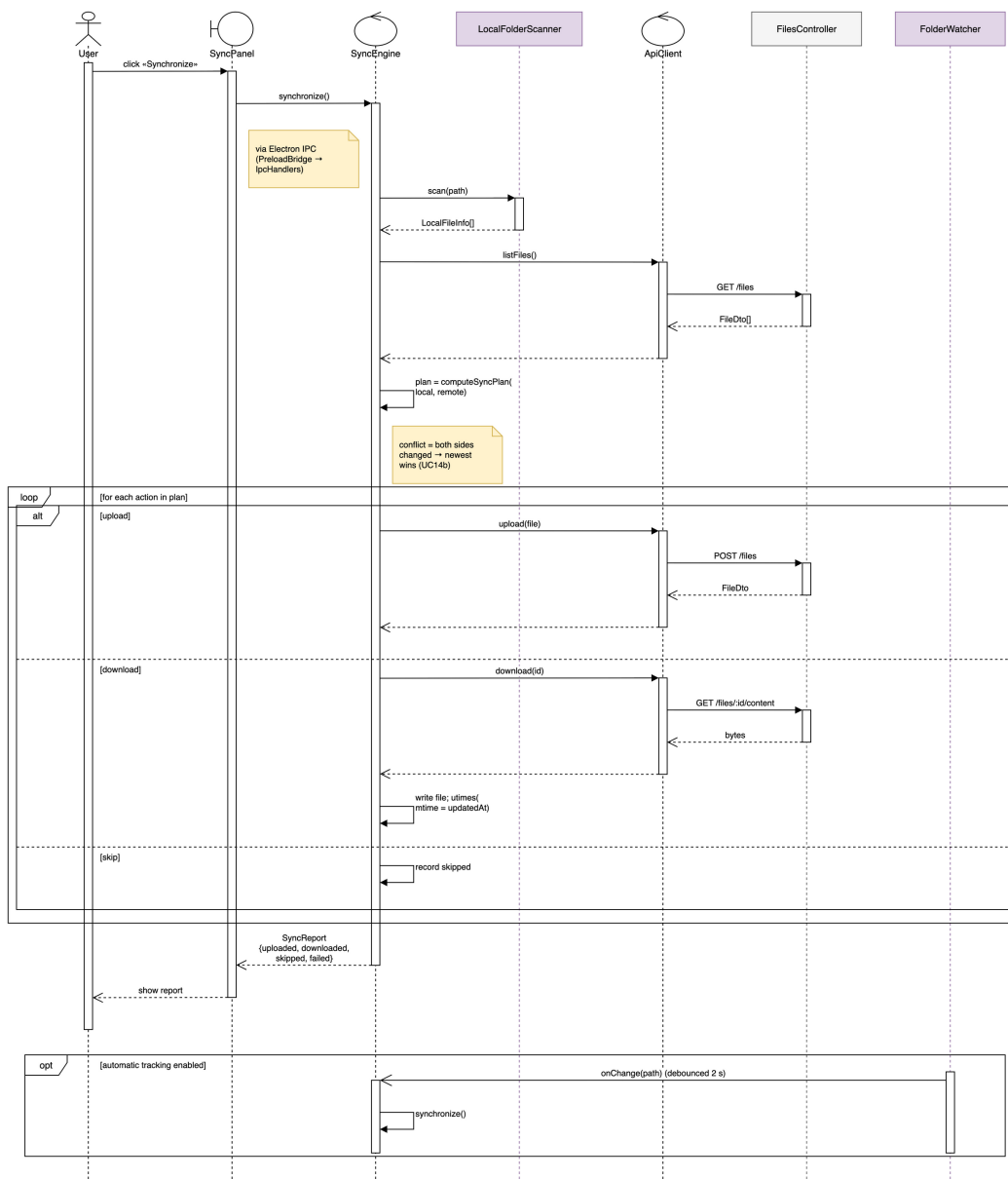
validateUser, пошук користувача та перевірка пароля verify(password, passwordHash); у гілці [valid] підписується JWT, ApiClient зберігає токен у SessionStore, і сторінка переходить до кабінету /drive; у гілці [invalid] повертається 401 і показується повідомлення про невірні дані. Примітка вказує параметри токена (HS256, 24 год, заголовок Authorization: Bearer).

4.9 Рис. 11. Діаграма послідовності — завантаження та перегляд вмісту (UC9, UC8)



Пояснення. Перша частина: користувач перетягує файл `report.cs` у вікно; `DrivePage` → `DriveViewModel` → `ApiClient` → `POST /files`; `FileService.upsert` шукає запис за `{ownerId, name}` і у фрагменті `alt` або перезаписує об'єкт та оновлює `FileEntry` (UC10), або створює запис і кладе об'єкт. Після відповіді `201 FileDto` модель повторно запитує `GET /files` і викликає `render(visibleFiles)`. Друга частина: клік по рядку викликає `preview(entry)`; фабрика `createPreview(entry)` обирає `TextPreview` для `.cs`; через `GET /files/:id/content` сервер отримує потік з `StorageService.getObject` і повертає `text/plain`; `PreviewPanel.showText(content)` показує вміст. Примітка пояснює, що для `.jpg` той самий виклик повертає `image/jpeg`, а `ImagePreview` відображає зображення.

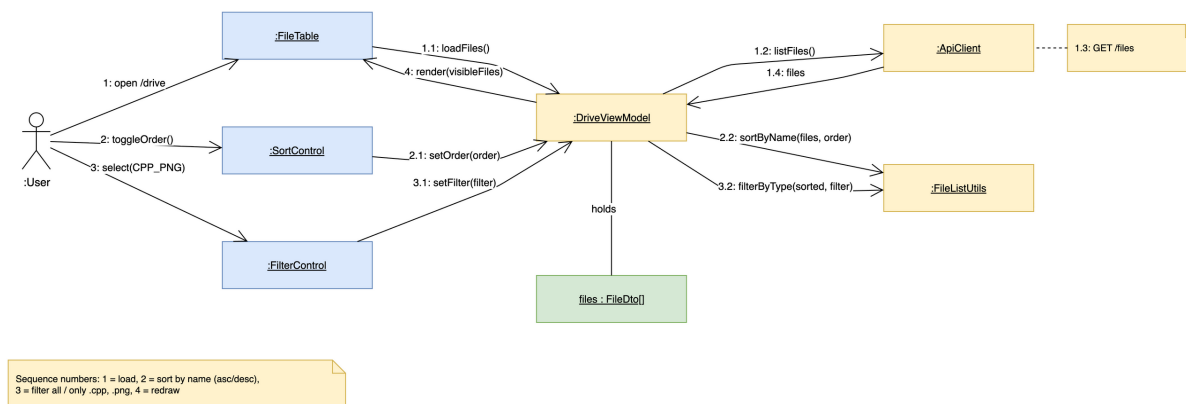
4.10 Рис. 12. Діаграма послідовності — синхронізація (UC14)



Пояснення. `SyncPanel` через IPC викликає `SyncEngine.synchronize()`. Рушій отримує `Local-`

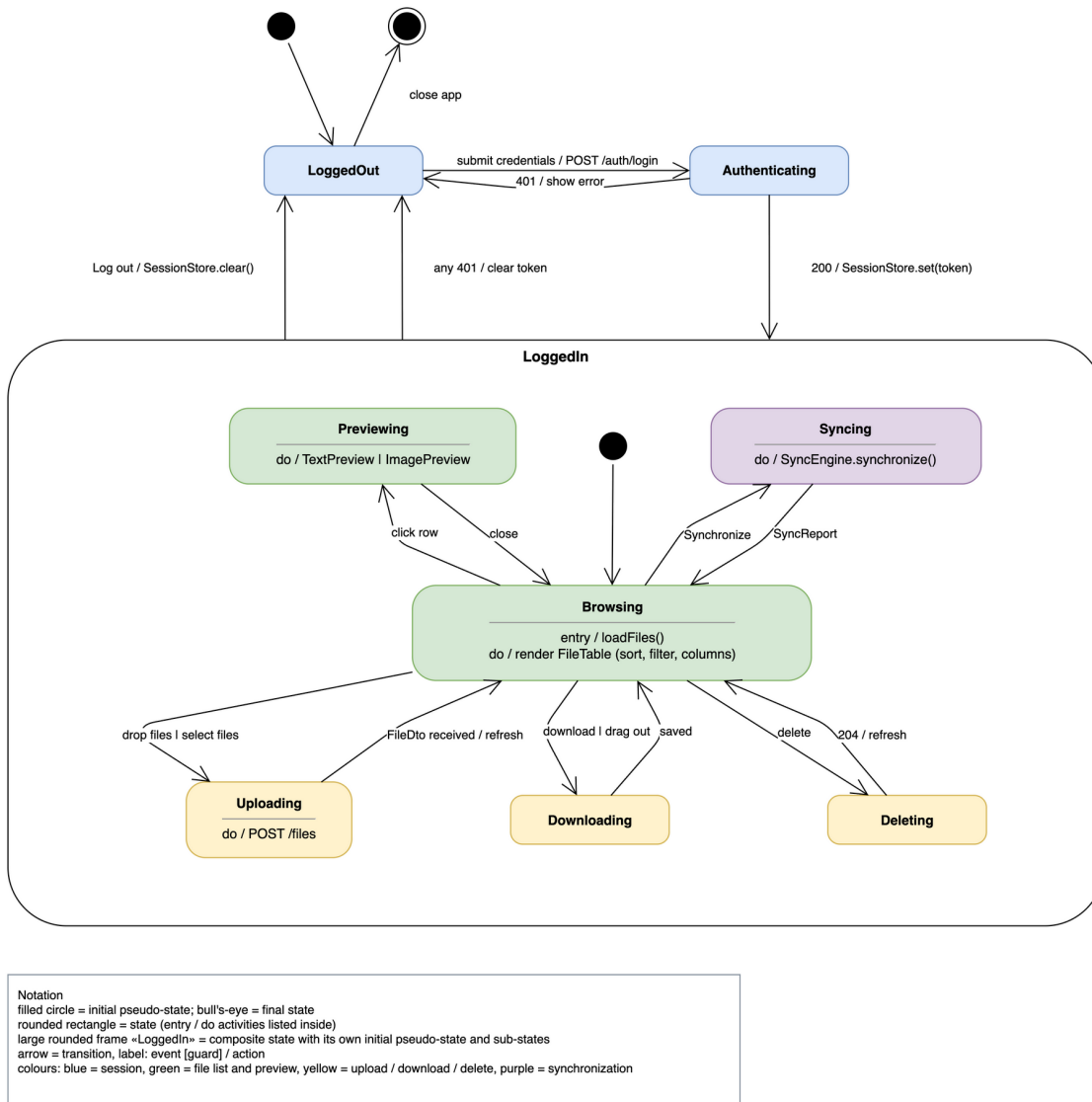
`FileInfo[]` від `LocalFolderScanner.scan(path)` і `FileDto[]` через `ApiClient.listFiles()` → GET `/files`, після чого обчислює план (`computeSyncPlan`). У фрагменті loop `[for each action in plan]` вкладений alt показує три гілки: `upload(file)` → POST `/files`, `download(id)` → GET `/files/:id/content` із записом файлу та встановленням `mtime`, або `skip`. Наприкінці повертається `SyncReport {uploaded, downloaded, skipped, failed}`, який панель показує користувачеві. Фрагмент opt `[automatic tracking enabled]` демонструє, як `FolderWatcher` через дебаунс 2 с ініціює повторну синхронізацію (UC14a). Примітка формулює правило конфлікту: змінено з обох боків — перемагає новіша версія (UC14b).

4.11 Рис. 13. Діаграма комунікації — сортування та фільтрація (UC5, UC6)



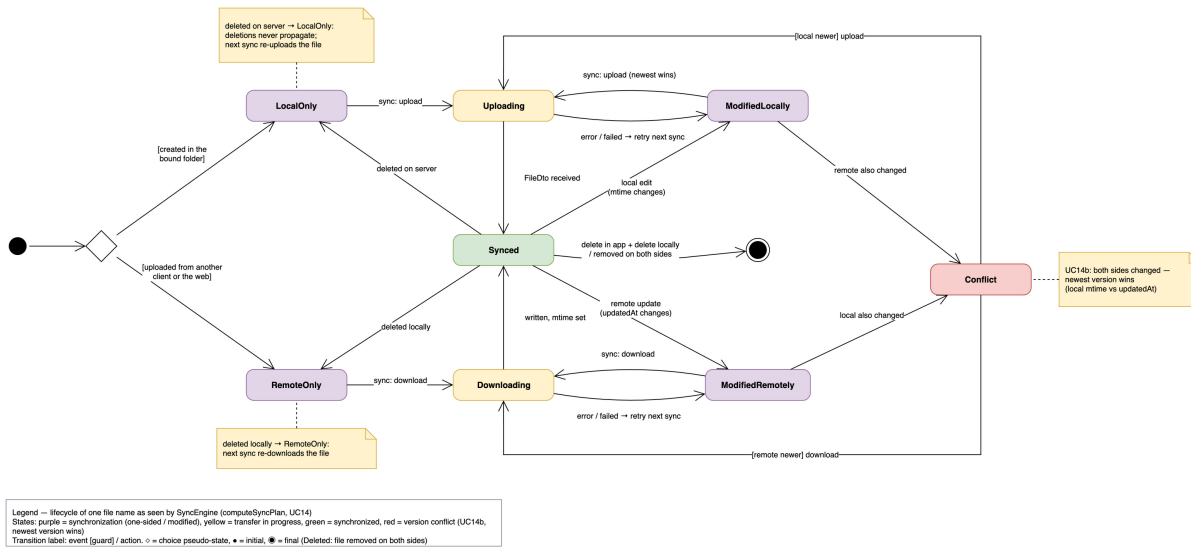
Пояснення. Діаграма комунікації є другим поданням взаємодії, що відповідає VOPC (рис. 5), і використовує ті самі об'єкти: `:FileTable`, `:SortControl`, `:FilterControl`, `:DriveViewModel`, `:ApiClient`, `:FileListUtils` та `files : FileDto[]`. Нумерація повідомлень задає порядок: 1 — відкриття кабінету та `loadFiles()`, 1.2–1.4 — отримання списку через `listFiles()` / GET `/files`; 2 — `toggleOrder()` на елементі сортування та 2.2 — `sortByName(files, order)`; 3 — `select(CPP_PNG)` на елементі фільтра та 3.2 — `filterByType(sorted, filter)`; 4 — `render(visibleFiles)`. Так на одній діаграмі видно обидві операції варіанта і те, що вони не потребують звернень до сервера.

4.12 Рис. 14. Діаграма станів — сеанс клієнта



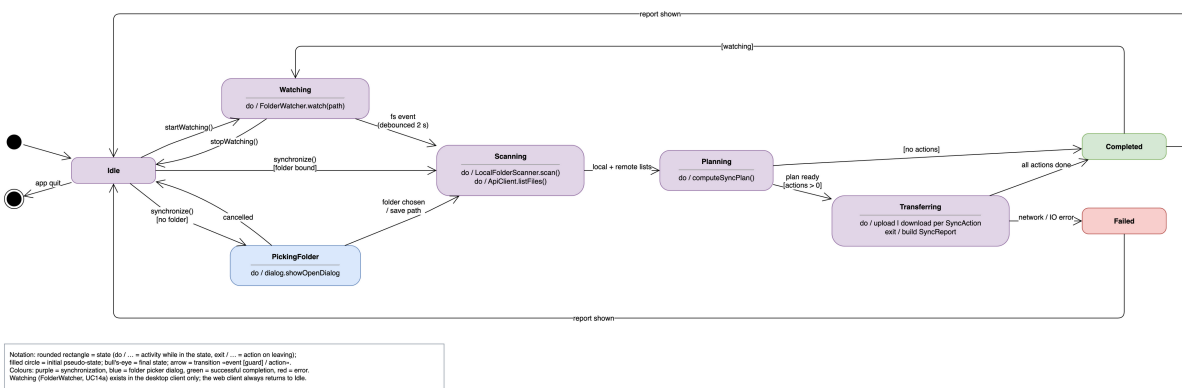
Пояснення. Клієнт стартує в стані `LoggedOut`; подання облікових даних переводить його в `Authenticating` (`POST /auth/login`), звідки за `401` він повертається назад, а за `200` із збереженням токена входить у складений стан `LoggedIn`. Усередині стан `Browsing` (entry / `loadFiles()`, do / відображення таблиці з сортуванням, фільтром і стовпцями) є центральним: клік по рядку веде до `Previewing`, кидання файлів — до `Uploading` (`POST /files`), вивантаження або перетягування з вікна — до `Downloading`, видалення — до `Deleting`, натискання «Synchronize» — до `Syncing`; кожен із цих станів після завершення повертає до `Browsing`. З будь-якого підстану вихід із системи або відповідь `401` переводить клієнт у `LoggedOut`; закриття застосунку — у кінцевий стан.

4.13 Рис. 15. Діаграма станів — життєвий цикл файлу під час синхронізації



Пояснення. Діаграма описує один файл (за назвою) очима SyncEngine. Файл з'являється або як LocalOnly (створений у прив'язаній папці), або як RemoteOnly (завантажений з іншого клієнта чи вебу); синхронізація переводить його через Uploading або Downloading у Synced. Локальна зміна (mtime) дає ModifiedLocally, віддалена (updatedAt) — ModifiedRemotely; якщо змінено обидві сторони, файл потрапляє у Conflict, який розв'язується правилом «новіша версія перемагає» — завантаженням або вивантаженням. Помилка передачі повертає файл до відповідного «modified»-стану з повторною спробою на наступній синхронізації. Видалення на сервері переводить файл у LocalOnly, видалення локально — у RemoteOnly: видалення не поширюються, і наступна синхронізація відновлює файл на другій стороні. Кінцевий стан досягається лише після видалення з обох боків.

4.14 Рис. 16. Діаграма станів — сеанс синхронізації (SyncEngine)



Пояснення. SyncEngine перебуває в Idle. Виклик synchronize() без прив'язаної папки веде до PickingFolder (діалог вибору), з папкою — одразу до Scanning (do / LocalFolderScanner.scan(), ApiClient.listFiles()), далі до Planning (do / computeSyncPlan()). Якщо дій немає — стан Completed; інакше Transferring (do / upload або download для кожної SyncAction, exit / побудова

SyncReport), звідки за успіху — Completed, за мережевою або файловою помилкою — Failed; в обох випадках після показу звіту — повернення в Idle. Стан Watching (do / FolderWatcher.watch(path)) існує лише в десктоп-клієнті: подія файлової системи з дебаунсом 2 с запускає Scanning, а після Completed при увімкненому відстеженні рушій повертається у Watching.

5 Інструменти

Діаграми створено як нативні файли draw.io (docs/uml/drawio/*.drawio), які можна редагувати в застосунку draw.io або в VS Code. Файли генеруються Python-скриптами (docs/uml/gen/*.py) за допомогою власної бібліотеки tools/drawio_gen; розкладку вузлів і маршрути зв'язків обчислює Graphviz, а PNG/SVG експортуються настільним застосунком draw.io. Такий підхід забезпечує єдині назви класів, компонентів і станів на всіх діаграмах та узгодженість із проєктною специфікацією.

6 Висновки

На етапі 1 виконано аналіз вимог і побудовано UML-специфікацію системи MiniDrive для варіанта 4-6. Визначено 15 прецедентів, які покривають усі загальні вимоги до системи типу 2, операції варіанта (сортування за назвою, фільтр .cpp/.png), типи варіанта (.cs як текст, .jpg як зображення) та обидва бонуси. Спроектowano архітектуру з одним REST-сервером (NestJS, PostgreSQL, MinIO), десктоп-клієнтом (Electron) і веб-клієнтом (Next.js) зі спільним пакетом логіки.

Вид діаграми	Вимога	Побудовано	Рисунки
прецедентів	1+	1	3
класів (одна — VOPC)	2+	4	4, 5 (VOPC), 6, 7
активності	1+	2	8, 9
взаємодії	2+	4	10, 11, 12 (послідовності), 13 (комунікації)
станів	1+	3	14, 15, 16
компонентів	1+	1	1
розгортання	1+	1	2

Отримана специфікація є основою етапу 2: реалізації REST-сервера та десктоп-клієнта з unit-тестами функцій sortByName і filterByType (операція варіанта), previewKindOf, toggleColumn і computeSyncPlan.